

PyieLink Framework

Architecture & Protocol Specification

Complete Reference: Vision, Architecture, Protocol Family, Cryptography, NAT Traversal, Application Services,
CLI Reference and Implementation Notes

Reference Implementation: **emtypyie-cli** version 3.0.1
(C implementation and Node.js implementation)

Author: EMTYPYIE
Contact: workforenty@gmail.com
Website: emtypyie.in

Copyright emtypyie.in

Notice

This document is the single, canonical specification for the PyieLink Framework. It consolidates architecture, protocol, cryptography, networking, application-layer, and command-line reference material into one continuously maintained file. Section numbers and terminology introduced here (PLSP, PLTP, VDS, PFTP, PRSP) are used consistently throughout and should be treated as the authoritative names for each subsystem in future revisions and in the codebase.

Abstract

PyieLink Framework is an open-source, peer-to-peer connectivity framework for establishing direct, authenticated, and encrypted channels between two devices with minimal dependence on always-on infrastructure. A lightweight, ephemeral signaling service is used only to introduce peers and exchange network candidates. Once a session is authenticated, session metadata is purged from the signaling server's memory and all further traffic moves onto a direct peer-to-peer lane, secured end-to-end with modern elliptic-curve cryptography. The framework targets use cases ranging from remote shell access and resumable file transfer to full remote desktop and cloud-gaming-style virtual machine streaming. This document describes the motivation, system architecture, protocol family, wire formats, cryptographic design, NAT traversal strategy, application-layer services, and the command-line interface shipped as **emtypyie-cli**, available today in both a C and a Node.js implementation.

Table of Contents

Notice	2
Abstract	2
Table of Contents	3
1. Introduction & Motivation	7
1.1 Problem Statement	7
1.2 Comparison with Existing Solutions	7
1.3 Design Philosophy	7
1.4 Document Scope	8
2. System Architecture	9
2.1 High-Level Overview	9
2.2 Component Breakdown	9
2.3 Control Plane vs. Data Plane	10
3. Open-Source Technology Stack	11
4. Protocol Family Overview	12
4.1 PLSP — Signaling Protocol	12
4.2 PLTP — Transport Protocol	12
4.3 VDS — Virtual Data Stream	12
4.4 PFTP — File Transfer Protocol	12
4.5 PRSP — Remote Shell Protocol	13
5. Session Lifecycle & Signaling Flow	14

5.1 Phase I — Registration	14
5.2 Phase II — Connect & Handoff	14
5.3 Phase III — Private Lane	14
5.4 Failure Handling	14
6. Wire Format Specification	15
6.1 Common PLTP Packet Header	15
6.2 Packet Type Enumeration	15
6.3 PLSP Signaling Messages (JSON over WebSocket)	16
6.4 Channel ID Allocation	16
7. Cryptographic Design	17
7.1 Identity Keys — Ed25519	17
7.2 Session Keys — X25519	17
7.3 Authenticated Encryption — ChaCha20-Poly1305	17
7.4 Forward Secrecy and Key Rotation	17
7.5 Threat Model	17
8. NAT Traversal Strategy	18
8.1 STUN	18
8.2 ICE Candidate Types	18
8.3 UDP Hole Punching Mechanics	18
8.4 Symmetric NAT and Relay Fallback	18
8.5 Keep-Alive	18
9. Transport Reliability & Congestion Control	19

9.1 Sequencing and Acknowledgement	19
9.2 Retransmission Strategy	19
9.3 Congestion Window	19
9.4 Channel Multiplexing Model	19
10. Application Layer Services	20
10.1 Remote Shell (PRSP)	20
10.2 File Transfer (PFTP)	20
10.3 Clipboard Synchronization	20
10.4 Remote Desktop and VM Streaming	20
10.5 Audio Forwarding	20
11. Command-Line Interface Reference — emtypyie-cli	21
11.1 Overview	21
11.2 Choosing Between the C and Node.js Builds	21
11.3 Command Reference	21
11.4 Example Session	22
12. Reference Implementation Notes	23
12.1 C Implementation — Packed Wire Header	23
12.2 Node.js Implementation — Ephemeral Signaling Switchboard	23
12.3 Cross-Implementation Parity	23
13. Security Considerations	24
14. Testing Strategy	25
14.1 Unit Level	25

14.2 Integration Level	25
14.3 Interop Testing	25
14.4 Security Testing	25
15. Performance Goals	26
16. Known Limitations & Open Engineering Questions	27
17. Roadmap	28
18. References	29
Appendix A — Full Byte-Level Wire Tables	30
A.1 PLTP Header (repeated for reference)	30
A.2 CHANNEL_OPEN Payload Layout	30
A.3 PFTP Chunk Header	30
Appendix B — Glossary	31

1. Introduction & Motivation

1.1 Problem Statement

Most commercial remote-access products (TeamViewer, AnyDesk, and comparable tools) route traffic through vendor-operated relay infrastructure that runs continuously, whether or not a session is active. This creates three structural problems: recurring operational cost for the operator, a permanent point of interception for every session, and a single point of failure that is entirely outside the control of the two people actually communicating. PyieLink Framework starts from a different premise: the server's only job is introduction, not carriage. Once two devices know how to reach each other and have verified each other's identity, the server has nothing left to do.

1.2 Comparison with Existing Solutions

The table below summarizes how PyieLink's approach differs from common existing tools. This is a design comparison, not a claim of superiority in every dimension — each of these tools was built for a different primary use case.

Tool	Relay Model	Identity Model	Primary Use Case
SSH	Direct (usually)	Host key / password / cert	Remote shell, tunneling
TeamViewer AnyDesk	/ Always-on vendor relay	Vendor account	Remote desktop support
Tailscale	Coordination server + optional relay (DERP)	WireGuard keys via identity provider	Private mesh networking
WireGuard	None (static config)	Curve25519 keypair	VPN tunnel
libp2p	DHT + relay peers	Peer ID from public key	Generic P2P applications
PyieLink Framework	Ephemeral signaling only, RAM-only, self-wipes	Ed25519 identity + X25519 session keys	Remote shell, files, desktop, VM streaming

1.3 Design Philosophy

- **Peer-to-peer first** — the signaling server never carries application data, only handshake metadata.
- **Zero-trust identity** — every device is identified by a public key, not a username and password pair.
- **Ephemeral signaling** — session state lives in RAM only and is deleted immediately after handoff.
- **Protocol family, not a monolith** — signaling, transport, and application concerns are separated so each can evolve independently.
- **Reference and production implementations** — a Node.js implementation for clarity and rapid iteration, and a C implementation for production performance, both conforming to the same wire specification.
- **Fully open-source dependencies** — no proprietary SDKs or closed relay services anywhere in the stack.

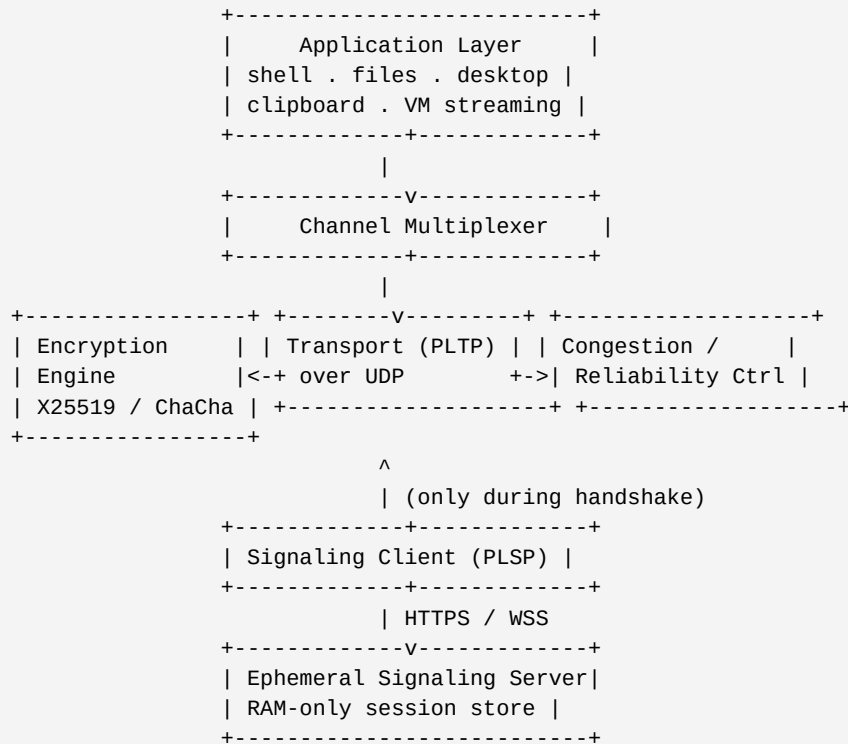
1.4 Document Scope

This specification covers the full protocol family, the cryptographic design, NAT traversal, the wire formats for every packet type currently defined, the planned application-layer services, and the command-line interface. Later chapters include reference pseudocode in both C and Node.js so that either implementation can be validated against the same behavior.

2. System Architecture

2.1 High-Level Overview

The framework separates the **control plane** (discovery, identity, session negotiation) from the **data plane** (encrypted application traffic). Applications built on top of PyieLink — remote shell, file transfer, remote desktop — never touch raw sockets directly; they consume a small internal API exposed by the framework's core modules.



2.2 Component Breakdown

Session Manager. Tracks local connection state, active channels, and session lifetime; the only module that knows the full state machine of a session.

Authentication Manager. Validates Ed25519 identity signatures during signaling and rejects any handoff that fails verification.

Transport Layer (PLTP). Frames outgoing data into PLTP packets, tracks sequence numbers, and hands incoming packets to the multiplexer.

Channel Multiplexer. Routes multiple logical streams — shell, file transfer, desktop — over a single PLTP transport connection using channel IDs.

Encryption Engine. Performs the X25519 key agreement and ChaCha20-Poly1305 authenticated encryption/decryption for every packet.

Congestion Controller. Adapts send rate based on observed loss and RTT so PyieLink behaves politely alongside other traffic on the same link.

Plugin Manager. Loads optional application modules (future: mesh routing, additional stream types) without changing the core transport.

2.3 Control Plane vs. Data Plane

The control plane exists only at the edges of a session's lifetime: once at startup to register, and once at connection time to exchange candidates. The data plane exists for the entire duration of the session and never involves the signaling server. This separation is what allows the signaling server to run on free or low-cost infrastructure without becoming a bottleneck or a liability: it is never in the path of bulk data, and it never accumulates logs of what was transferred.

3. Open-Source Technology Stack

Every component in the recommended stack is open-source and widely used in production systems, which keeps the framework auditable and avoids vendor lock-in at any layer.

Layer	Component	Purpose
Signaling server	Node.js + uWebSockets.js	Lightweight WebSocket server; fast cold start on free-tier hosts; handles the HTTP-to-WS upgrade.
Session store	In-memory Map (RAM only)	Volatile session metadata with explicit deletion on handoff; no disk writes, ever.
Cryptography	libsodium (C) / sodium-native (Node.js)	Ed25519 identity signatures, X25519 key exchange, ChaCha20-Poly1305 AEAD.
NAT traversal	libjuice / libdatachannel	ICE agent implementation for STUN-based candidate gathering and UDP hole punching.
STUN servers	Public STUN (e.g. stun.l.google.com)	Public IP/port discovery without requiring the signaling server to stay online.
CLI — production	C (POSIX sockets)	Low-overhead, low-latency implementation; static binaries for easy distribution.
CLI — reference	Node.js	Fast iteration and a readable reference implementation of the protocol.
Streaming (optional)	FFmpeg, Sunshine / Moonlight	Reusable open-source pipeline for VM and desktop video streaming over the private lane.
Hosting	Any free/low-cost web-service host (signaling only)	Ephemeral web service; never part of the data path, only introduces peers.

4. Protocol Family Overview

Rather than a single monolithic protocol, PyieLink defines a small family of cooperating protocols, each responsible for one concern. This keeps the signaling protocol simple and lets application protocols evolve independently of the transport.

Name	Full Name	Responsibility
PLSP	PyieLink Signaling Protocol	Peer discovery, identity exchange, ICE candidate relay, ephemeral session bootstrapping.
PLTP	PyieLink Transport Protocol	Encrypted packet transport, sequencing, retransmission, and channel multiplexing over UDP.
VDS	Virtual Data Stream	Logical stream abstraction used by applications for screen, audio, and interactive data.
PFTP	PyieLink File Transfer Protocol	Resumable, chunked file upload and download over a VDS channel.
PRSP	PyieLink Remote Shell Protocol	PTY allocation and forwarding, including terminal resize events.

4.1 PLSP — Signaling Protocol

PLSP runs over WebSockets so that it works through the HTTPS-only ports exposed by most free-tier hosting platforms. Every PLSP message is a small JSON object; brevity here matters because these messages are only ever exchanged during the few seconds of session setup.

4.2 PLTP — Transport Protocol

PLTP is the binary protocol that carries all data-plane traffic once the private lane is established. It provides sequencing, optional retransmission for control-sensitive channels, and light congestion control, while remaining thin enough to sustain high-throughput video streams when reliability is not required for every packet.

4.3 VDS — Virtual Data Stream

A VDS is a logical channel multiplexed over a single PLTP connection, identified by a small integer channel ID. Multiple VDS channels can be open simultaneously — for example a shell channel, a file-transfer channel, and a clipboard channel all running over one underlying UDP transport.

4.4 PFTP — File Transfer Protocol

PFTP splits files into fixed-size chunks, each independently acknowledged, so an interrupted transfer can resume from the last confirmed chunk rather than restarting from zero.

4.5 PRSP — Remote Shell Protocol

PRSP forwards a pseudo-terminal (PTY) session, including window-resize events, over a dedicated VDS channel, giving the client an experience equivalent to a local interactive shell.

5. Session Lifecycle & Signaling Flow

5.1 Phase I — Registration

- Host runs **relay start**. The CLI sends an HTTPS request to the signaling service, which wakes it if it is asleep (typical free-tier cold start: 30 to 50 seconds).
- Once awake, the host opens a WebSocket connection and registers its identity as **username@public_key** along with its gathered network candidates.
- The server stores this in RAM only and starts a countdown (for example, 4 minutes) aligned with the host's inactivity spin-down window.

5.2 Phase II — Connect & Handoff

- Client runs **relay connect <username@public_key>**, opening its own WebSocket session to the signaling server.
- The server exchanges each side's public key and ICE candidates with the other.
- As soon as both sides confirm receipt, the server deletes the session record from memory.
- The signaling server is no longer involved; it returns to idle and eventually spins down.

5.3 Phase III — Private Lane

- Both peers attempt direct UDP hole punching using the exchanged candidates.
- Once a path is open, peers run an X25519 ECDH handshake directly over that path to derive a shared session key.
- All further traffic is authenticated and encrypted end-to-end and never touches the signaling server.
- Lightweight heartbeat packets keep NAT and firewall mappings alive during idle periods.

```

Host                Signaling Server                Client
|--- relay start ----->|
|<-- online, 4:00 window -----|
|                |<----- relay connect -----|
|<----- candidates(Client) ----|----- candidates(Host) ----->|
|                [server wipes session from RAM]                |
|===== UDP hole punching =====|
|===== X25519 handshake =====|
|===== Encrypted private lane =====|

```

5.4 Failure Handling

If the signaling window expires before a client connects, the host must run **relay start** again. If hole punching does not succeed within a bounded number of attempts (see Chapter 8), the CLI reports a clear failure rather than hanging indefinitely, since there is currently no always-on relay to fall back to.

6. Wire Format Specification

6.1 Common PLTP Packet Header

All data-plane traffic on the private lane is carried inside a fixed 17-byte header followed by a variable-length authenticated ciphertext payload:

byte offset	field	size	notes
0 - 1	magic_bytes	2 B	constant 0x50 0x32 ("P2")
2	packet_type	1 B	see Section 6.2
3 - 4	payload_length	2 B	big-endian, length of ciphertext
5 - 16	nonce	12 B	unique IV for ChaCha20-Poly1305
17 ...	ciphertext	var	AEAD-encrypted application payload

6.2 Packet Type Enumeration

Value	Name	Meaning
0x01	REGISTER	Sent to the signaling server during Phase I registration.
0x02	HANDSHAKE	Carries identity signature and ICE candidate data during Phase II.
0x03	HOLE_PUNCH	Probe packet used purely to open NAT/firewall mappings.
0x04	HEARTBEAT	Small keep-alive packet sent on an idle private lane.
0x05	DATA	Encrypted application payload for any active VDS channel.
0x06	CHANNEL_OPEN	Requests a new logical VDS channel with a given channel ID and type.
0x07	CHANNEL_CLOSE	Signals that a VDS channel is finished and its ID may be reused.

6.3 PLSP Signaling Messages (JSON over WebSocket)

```
// REGISTER – host to signaling server
{
  "type": "REGISTER",
  "username": "zeus",
  "publicKey": "ed25519:9f2a...",
  "candidates": ["203.0.113.10:51820", "192.168.1.14:51820"]
}

// CONNECT – client to signaling server
{
  "type": "CONNECT",
  "targetUser": "zeus@ed25519:9f2a...",
  "publicKey": "ed25519:71bd...",
  "candidates": ["198.51.100.7:51820"]
}

// HANDOFF – signaling server to both peers, then session is wiped
{
  "type": "HANDOFF",
  "peerPublicKey": "ed25519:...",
  "peerCandidates": ["..."]
}
```

6.4 Channel ID Allocation

Channel IDs are 16-bit values assigned by the peer that opens the channel. Even values are reserved for host-initiated channels and odd values for client-initiated channels, which avoids collisions without requiring a negotiation round trip.

7. Cryptographic Design

7.1 Identity Keys — Ed25519

Each device generates a long-term Ed25519 signing keypair on first run. The public key (or a short hash of it) is the device's address, used in commands such as **relay connect username@public_key**. The private key never leaves the device and is stored with restrictive file permissions.

7.2 Session Keys — X25519

An ephemeral X25519 Elliptic Curve Diffie-Hellman exchange, run directly between peers after the signaling handoff, derives a shared secret that never crosses the signaling server. Because each side mixes its own private key with the other side's public key, both sides arrive at an identical shared secret without ever transmitting it.

7.3 Authenticated Encryption — ChaCha20-Poly1305

Every PLTP data packet is encrypted and integrity-checked with ChaCha20-Poly1305. If a packet is modified in transit, decryption fails and the packet is silently dropped rather than being processed. Nonces are derived from a monotonic per-session counter to guarantee uniqueness for the lifetime of the session key.

7.4 Forward Secrecy and Key Rotation

Because session keys are ephemeral and derived fresh for every connection, compromise of a long-term identity key does not expose the content of past sessions. For long-running sessions, the framework should re-run the X25519 exchange periodically (for example, hourly) and rotate the active session key, limiting the amount of data protected by any single key.

7.5 Threat Model

- **In scope:** a network-level attacker observing or modifying traffic between the two peers or between a peer and the signaling server.
- **In scope:** a compromised or malicious signaling server attempting to impersonate a peer or read session content.
- **Out of scope:** a compromised endpoint device — if the host machine itself is compromised, no transport-layer protocol can protect it.
- **Out of scope (current version):** traffic analysis / metadata leakage such as packet timing and size, which is not yet addressed by padding or timing obfuscation.

Design note: because the signaling server only ever sees public keys and network candidates, even a fully compromised signaling server cannot decrypt session traffic or derive the shared secret — it can, at most, attempt to introduce two peers that never intended to connect, which identity verification via Ed25519 signatures is specifically designed to prevent.

8. NAT Traversal Strategy

8.1 STUN

Each device first queries a public STUN server to discover its own public IP address and the port a NAT device is currently mapping its outbound traffic to. This step requires no involvement from the PyieLink signaling server.

8.2 ICE Candidate Types

Candidate Type	Description
Host	The device's own local network address, useful when both peers are on the same LAN.
Server Reflexive	The public address and port observed by a STUN server, used for most NAT-to-NAT connections.
Relayed	An address on a TURN-style relay, used only if direct connection attempts fail (see 8.4).

8.3 UDP Hole Punching Mechanics

Once both sides have exchanged candidates, they simultaneously send UDP packets toward each other's public endpoint. Each outbound packet causes the sender's NAT to open a temporary mapping; because both sides send at roughly the same time, the returning packet from the other peer typically arrives while that mapping is still open, and a bidirectional path is established without either side needing to accept an unsolicited inbound connection.

8.4 Symmetric NAT and Relay Fallback

On strict corporate or public networks, routers may use a symmetric NAT that assigns a different external port for every destination, which can cause hole punching to fail. Because the free-tier signaling server does not stay online long enough to act as a data relay, this version of the specification treats a failed hole-punch attempt as a reportable error rather than silently degrading to a relay. A future revision (see Chapter 16, Roadmap) may add a minimal, explicitly-invoked TURN-style relay for this case.

8.5 Keep-Alive

Once a private lane is open, a small HEARTBEAT packet (0x04) is sent periodically (for example, every 20 seconds) during idle periods so that the NAT/firewall mapping on both sides does not expire and force a fresh hole-punch.

9. Transport Reliability & Congestion Control

9.1 Sequencing and Acknowledgement

Each PLTP data packet on a reliable channel (such as PRSP or PFTP) carries a monotonically increasing sequence number. The receiver periodically sends a compact acknowledgement listing the highest contiguous sequence number received and a bitmap of recently received out-of-order packets, similar in spirit to TCP SACK.

9.2 Retransmission Strategy

Packets that are not acknowledged within an adaptive timeout, computed from a smoothed round-trip-time estimate, are retransmitted. Channels that tolerate loss — most notably video streaming for remote desktop and VM access — can opt out of retransmission entirely and rely on the underlying codec's error concealment instead.

9.3 Congestion Window

A simple additive-increase, multiplicative-decrease scheme governs how many unacknowledged packets may be outstanding at once, so that a PyieLink session shares link capacity fairly with other traffic on the same network rather than saturating it.

9.4 Channel Multiplexing Model

All logical channels share one PLTP connection and one congestion window, but each channel has its own independent sequence-number space. This keeps a slow file transfer from blocking an interactive shell channel running over the same underlying connection.

10. Application Layer Services

10.1 Remote Shell (PRSP)

The host allocates a pseudo-terminal (PTY) and forwards its input and output over a dedicated VDS channel. Terminal resize events are sent as small control messages on the same channel so that full-screen terminal applications render correctly on the client.

10.2 File Transfer (PFTP)

Files are split into fixed-size chunks (a typical default is 64 KB) and streamed over a dedicated channel. Each chunk carries its own index, so an interrupted transfer resumes by requesting only the missing chunk indices rather than restarting from the beginning.

10.3 Clipboard Synchronization

A lightweight control channel mirrors clipboard content between host and client on change, gated by an explicit user-configurable setting since clipboard sync has meaningful privacy implications.

10.4 Remote Desktop and VM Streaming

Rather than inventing a new video codec or streaming protocol, PyieLink reuses existing open-source pipelines: FFmpeg for encoding, and Sunshine/Moonlight-style protocols for the actual interactive streaming session. PyieLink's job ends at establishing the secure, direct transport; the streaming pipeline runs on top of it.

10.5 Audio Forwarding

Audio is treated as a separate, loss-tolerant VDS channel, encoded with an open-source codec (such as Opus) chosen for low latency rather than maximum compression.

11. Command-Line Interface Reference — emtypyie-cli

11.1 Overview

The reference command-line tool is **emtypyie-cli**, currently at version **3.0.1**, shipped as two separate, independently maintained implementations: one written in C and one written in Node.js. Both implement the same PLSP/PLTP wire specification described in this document, so a host running one implementation can connect to a client running the other without any compatibility issue — the wire format, not the implementation language, is the contract between them.

11.2 Choosing Between the C and Node.js Builds

For a single canonical build to standardize on for day-to-day and production use, the C implementation is the better default. The reasoning:

- **Lower latency and overhead** — no JavaScript runtime or garbage collector sits between the CLI and the socket layer, which matters for hole punching timing and for high-throughput channels like VM streaming.
- **Smaller, dependency-free static binaries** — easier to distribute to a target/host machine that may not already have Node.js installed.
- **Lower idle resource usage** — relevant when the CLI runs continuously on a host machine waiting for **relay connect** from a client.

The Node.js build remains valuable as the reference implementation for validating protocol changes quickly and for scripting/tooling around the CLI, but it is recommended as a development and testing tool rather than the default for end users. Keeping both aligned to the same wire specification (this document) is what makes that split safe: either build can be swapped in without breaking compatibility with a peer running the other one.

11.3 Command Reference

Command	Description
relay start	Wakes the signaling service, registers this device's identity and candidates, and opens the connection window.
relay connect <user@pubkey>	Requests introduction to the named peer and attempts to establish the private lane.
relay status	Shows whether a session is currently registered, connected, or idle, and the remaining window time.
relay stop	Tears down the active private lane and, if registered, de-registers from the signaling service.
relay keygen	Generates a new Ed25519 identity keypair for this device.
relay whoami	Prints this device's current public identity in username@publickey form.

11.4 Example Session

```
# On the host / target machine
$ relay start
[+] Waking signaling service...
[+] Registered as zeus@ed25519:9f2a...
[+] Online -- connection window closes in 04:00

# On the client / controlling machine
$ relay connect zeus@ed25519:9f2a...
[+] Contacting signaling service...
[+] Peer found, exchanging candidates and identity signatures
[+] Handshake verified, disconnecting from signaling service
[+] Punching NAT toward peer...
[+] Private lane established
```

12. Reference Implementation Notes

12.1 C Implementation — Packed Wire Header

```
#include <stdint.h>

typedef struct __attribute__((packed)) {
    uint16_t magic_bytes; /* 0x5032 */
    uint8_t packet_type; /* see Section 6.2 */
    uint16_t payload_len; /* big-endian */
    uint8_t nonce[12];    /* ChaCha20-Poly1305 IV */
} pyielink_header_t;

void on_ice_state_change(juice_agent_t *agent, juice_state_t state) {
    if (state == JUICE_STATE_COMPLETED) {
        /* drop signaling socket, derive session key via X25519,
           and switch packet processing to the private lane */
    }
}
```

12.2 Node.js Implementation — Ephemeral Signaling Switchboard

```
const uWS = require('uWebSockets.js');
const sessions = new Map(); // RAM only, never persisted

uWS.App().ws('/signal', {
  message: (ws, message) => {
    const msg = JSON.parse(Buffer.from(message).toString());
    if (msg.type === 'REGISTER') {
      sessions.set(msg.username, { ws, netInfo: msg.netInfo });
    } else if (msg.type === 'CONNECT') {
      const target = sessions.get(msg.targetUser);
      if (target) {
        target.ws.send(JSON.stringify({ type: 'HANDOFF', peer: msg.netInfo }));
        ws.send(JSON.stringify({ type: 'HANDOFF', peer: target.netInfo }));
        sessions.delete(msg.targetUser); // wipe on handoff
      }
    }
  }
}).listen(8080, (token) => {
  if (token) console.log('Signaling service online');
});
```

12.3 Cross-Implementation Parity

Both implementations must be validated against the same conformance checklist: identical packet header layout, identical packet type enumeration, identical nonce construction, and identical channel ID allocation rules. A small conformance test suite that sends known input vectors through each implementation's encoder/decoder and compares raw bytes is the most reliable way to keep the two implementations from silently diverging as features are added.

13. Security Considerations

- Identity private keys must be stored with restrictive filesystem permissions and never transmitted, logged, or included in diagnostic output.
- The signaling server must never write session data to disk, swap, or any persistent log; all session state should be held in structures that are explicitly cleared (not just dereferenced) after handoff.
- Every HANDSHAKE message must be signature-verified before its contents are trusted, to prevent the signaling server (or an attacker with access to it) from introducing an unverified peer.
- Nonce reuse under the same session key must be structurally prevented (monotonic counter, not random generation) since nonce reuse breaks ChaCha20-Poly1305's confidentiality guarantees.
- CLI builds should be reproducible and, where possible, distributed with checksums so a downloaded binary can be verified before it is trusted to hold identity keys.

14. Testing Strategy

14.1 Unit Level

Packet encoding/decoding, nonce construction, and channel ID allocation should each have direct unit tests in both the C and Node.js codebases, run against shared test vectors.

14.2 Integration Level

A local test harness should run a signaling server plus two CLI instances on a simulated NAT (using network namespaces or a local NAT-simulation tool) to exercise the full registration, handoff, and hole-punching flow without requiring real public internet access.

14.3 Interop Testing

Because two independent implementations exist, every release should include a run where the C build acts as host and the Node.js build acts as client, and vice versa, confirming that the wire specification — not implementation detail — is what peers actually agree on.

14.4 Security Testing

Fuzzing the packet parser (particularly `payload_length` handling and channel ID handling) and running a static analyzer over both codebases should be part of the standard release checklist.

15. Performance Goals

Metric	Target
Signaling handoff time (warm server)	Under 1 second from relay connect to HANDOFF delivery.
Signaling handoff time (cold server)	Under 60 seconds including free-tier cold start.
Hole-punch success rate (typical home NAT)	Above 90% within 3 attempts.
Private-lane round-trip overhead vs. raw UDP	Under 2 ms attributable to encryption/framing.
Idle heartbeat bandwidth	Under 1 KB/min per session.

These are engineering targets to validate against, not guarantees; actual figures depend heavily on network conditions and the chosen hosting tier for the signaling service.

16. Known Limitations & Open Engineering Questions

- Free-tier hosts typically expose HTTP/HTTPS only, so signaling must run over WebSockets rather than raw sockets.
- No relay fallback currently exists for symmetric-NAT cases where hole punching fails; Section 8.4 documents this as a deliberate current limitation rather than an oversight.
- Reconnect behavior after a dropped session is not yet fully specified — whether the host should automatically re-register or the client should hold an ephemeral key for fast re-punching remains an open decision.
- High-throughput use cases (VM streaming, cloud gaming) intentionally reuse an existing open-source media pipeline rather than a custom video protocol, which is the right tradeoff but does add an external dependency.
- Multi-peer and mesh scenarios are explicitly out of scope for this version of the specification.

17. Roadmap

- **v0.1** — Signaling server, identity/session handshake, basic encrypted transport (current scope of this document).
- **v0.2** — File transfer channel (PFTP) with resumable transfers.
- **v0.3** — Remote shell channel (PRSP) with terminal resize support.
- **v0.4** — Clipboard synchronization and basic remote desktop framing.
- **v0.5** — Relay fallback for restrictive NATs; connection quality diagnostics.
- **v1.0** — Plugin SDK, VM / cloud-gaming streaming integration, multi-peer mesh exploration.

18. References

- RFC 8445 — Interactive Connectivity Establishment (ICE).
- RFC 5389 — Session Traversal Utilities for NAT (STUN).
- RFC 8489 — STUN (updated).
- RFC 8446 — The Transport Layer Security (TLS) Protocol, Version 1.3 (design reference for handshake patterns).
- Bernstein, D. J. — Curve25519: New Diffie-Hellman Speed Records.
- Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., Yang, B.-Y. — High-Speed High-Security Signatures (Ed25519).
- Bernstein, D. J. — ChaCha, a variant of Salsa20.
- libsodium documentation — <https://doc.libsodium.org>
- libjuice (ICE library) — <https://github.com/paullouisageneau/libjuice>
- WireGuard whitepaper — J. Donenfeld.

Appendix A — Full Byte-Level Wire Tables

A.1 PLTP Header (repeated for reference)

byte offset	field	size	notes
0 - 1	magic_bytes	2 B	constant 0x50 0x32 ("P2")
2	packet_type	1 B	see Section 6.2
3 - 4	payload_length	2 B	big-endian, length of ciphertext
5 - 16	nonce	12 B	unique IV for ChaCha20-Poly1305
17 ...	ciphertext	var	AEAD-encrypted application payload

A.2 CHANNEL_OPEN Payload Layout

byte offset	field	size	notes
0 - 1	channel_id	2 B	even = host-initiated, odd = client-initiated
2	channel_type	1 B	0x01 shell, 0x02 file, 0x03 clipboard, 0x04 video, 0x05 audio
3 - 4	reserved	2 B	must be zero

A.3 PFTP Chunk Header

byte offset	field	size	notes
0 - 7	file_id	8 B	unique per transfer
8 - 11	chunk_index	4 B	big-endian, zero-based
12 - 15	chunk_length	4 B	big-endian, bytes in this chunk

Appendix B — Glossary

AEAD — Authenticated Encryption with Associated Data; encryption that also guarantees integrity.

ECDH — Elliptic Curve Diffie-Hellman; a method for two parties to derive a shared secret without transmitting it.

ICE — Interactive Connectivity Establishment; the framework for finding a usable network path between two peers.

NAT — Network Address Translation; the mechanism most routers use to share one public IP among multiple local devices.

PLSP — PyieLink Signaling Protocol.

PLTP — PyieLink Transport Protocol.

PFTP — PyieLink File Transfer Protocol.

PRSP — PyieLink Remote Shell Protocol.

STUN — Session Traversal Utilities for NAT; a protocol for discovering a device's public-facing address.

VDS — Virtual Data Stream; the logical channel abstraction used by PyieLink applications.